

Prototype Documentation

Link: [PDAI-A2-Recording.webm](#)

Utility of the Prototype:

- **Efficient Content Consumption:**

The tool automatically extracts transcripts from YouTube videos and generates a structured, professional summary. This allows users to quickly understand key points without watching the entire video.

- **Interactive Exploration:**

An integrated chatbot enables users to ask open-ended questions about the video content, fostering deeper engagement and personalized insights.

- **Enhanced Productivity:**

By combining automated summarization with a conversational Q&A interface, the prototype streamlines information extraction and learning, making it valuable for research, education, and professional use.

Main Design Decisions:

- **Transcript Retrieval:**

Leveraged the `youtube-transcript-api` to automatically fetch video transcripts.

- **Structured Summarization:**

Designed a multi-section summary prompt (including Title, Time Interval, Summary, Insights Based on Numbers, and Example Exploratory Questions) using LangChain's `ChatOpenAI` for a professional output inspired by the expectations document.

- **Persistent, Session-Based Chat Interface:**

Used Streamlit's session state and `StreamlitChatMessageHistory` to maintain the generated summary and ongoing conversation, ensuring the chat interface remains visible and context is preserved.

- **Modular Code Architecture:**

Separated the application into distinct modules (`app.py` for UI and main logic, and `chatbot.py` for conversation handling), which simplifies maintenance and future enhancements.

- **Custom UI Enhancements:**

Applied custom CSS styling to improve aesthetics, including modern fonts, color schemes, and responsive layout adjustments for a more professional look.

Main Difficulties Found:

- **Package Compatibility Issues:**

Conflicts arose between different versions of `openai`, `langchain`, `langchain-openai`, and `langsmith`. The resolution involved carefully pinning compatible versions and updating code to align with the new OpenAI API.

- **Migration Limitations on Windows:**

The migration CLI for OpenAI was not supported on Windows, which forced manual adjustments in the code to adopt the new API.

- **State Management in Streamlit:**

Ensuring the summary and chat interface remained persistent required careful management of Streamlit's session state to prevent UI elements from disappearing after interactions.

- **Method Signature Conflicts:**

Encountered errors related to the `StreamlitChatMessageHistory` methods, which were resolved by switching to `add_user_message` and `add_ai_message` instead of a generic `add_message`.

- **Prompt Tuning:**

Iterative refinements were necessary to achieve a structured summary that meets the professional style outlined in the expectations, balancing clarity with depth.